PROPOSTA DE TRABALHO

Ano letivo: 20 / 20

CURSO PROFISSIONAL DE Selecionar Curso

Disciplina / UFCD / UC	Módulo / Tema
Redes	**Programação de Sistemas de Comunicação**
Data	**Duração**
09-11-2026	**180 minutos**

1. TEMA / TÍTULO DO TRABALHO

API / Gestão de Tarefas

2. OBJETIVOS

Indique o que se pretende alcançar com a realização deste trabalho.

Demonstrar a capacidade de criar uma API que permite comunicar com sistemas externos

3. TAREFAS / DESENVOLVIMENTO

Descreve as etapas, questões ou atividades a realizar.

Instruções gerais

Uma empresa pretende desenvolver uma API REST para permitir a gestão de tarefas dos seus colaboradores. A aplicação deverá permitir o registo e autenticação de utilizadores, a criação e gestão de tarefas e o controlo de acessos de acordo com o tipo de utilizador autenticado.

A API deverá utilizar uma base de dados relacional, criada automaticamente através do SQLAlchemy, e implementar um sistema de autenticação baseado em tokens JWT (JSON Web Token).

O sistema deverá contemplar dois tipos de utilizadores:

- Admin: possui permissões de administração e pode consultar, alterar e eliminar todas as tarefas existentes.
- Utilizador: pode criar tarefas e consultar, alterar ou eliminar exclusivamente as suas próprias tarefas.

Estrutura da base de dados

A aplicação deverá criar automaticamente uma base de dados SQLite com duas tabelas relacionadas.

Tabela: Utilizador

Campo	Tipo	Descrição
id	Integer	Chave primária
username	String	Nome de utilizador
email	String	Endereço de email
password	String	Palavra-passe
tipo	String	admin ou utilizador

Tabela: Tarefa

Campo	Tipo	Descrição
id	Integer	Chave primária
titulo	String	Título da tarefa
descricao	Text	Descrição da tarefa
estado	String	Pendente, Em curso ou Concluída
utilizador_id	Integer	Chave estrangeira para Utilizador

Relacionamento: Um utilizador pode possuir várias tarefas. Cada tarefa pertence exclusivamente a um utilizador.

Requisitos Funcionais

Criação do projeto e base de dados

1. Criar um projeto Python para desenvolvimento da API.
2. Instalar e configurar as bibliotecas necessárias.
3. Criar a aplicação Flask.
4. Configurar o SQLAlchemy para utilizar uma base de dados SQLite.
5. Criar os modelos Utilizador e Tarefa.
6. Estabelecer o relacionamento entre as duas tabelas.
7. Criar automaticamente as tabelas na primeira execução da aplicação.

Registo e autenticação

Implementar os seguintes recursos:

1. Registo de utilizadores

Endpoint: POST /registo

Deverá permitir registar um novo utilizador que deve receber os seguintes dados:

- username;
- email;
- password.

Requisitos:

- Não permitir o registo de emails já existentes.
- Todos os novos registos deverão receber automaticamente o tipo utilizador.
- Não deverá ser possível escolher o tipo admin através do pedido de registo.

2. Autenticação

Endpoint: POST /login

Deverá receber o email e a palavra-passe do utilizador.

Após validar as credenciais, deverá devolver um token JWT.

O token deverá permitir identificar o utilizador autenticado e o respetivo tipo de acesso.

3. Criação de administrador (no app.py)

A aplicação deverá possuir, inicialmente, pelo menos um utilizador com o perfil admin.

Este utilizador poderá ser criado através de um procedimento de inicialização da aplicação, sem permitir que um utilizador comum se atribua privilégios de administrador.

Gestão de tarefas (CRUD)

Implementar os seguintes endpoints:

Método	Endpoint	Funcionalidade
POST	/tarefas	Criar uma tarefa
GET	/tarefas	Consultar tarefas
GET	/tarefas/id	Consultar uma tarefa específica
PUT	/tarefas/id	Alterar uma tarefa
DELETE	/tarefas/id	Eliminar uma tarefa

<u>Regras de funcionamento:</u>

- Um utilizador autenticado pode criar tarefas.
- Ao criar uma tarefa, esta deverá ficar automaticamente associada ao utilizador autenticado.
- Um utilizador comum apenas poderá consultar as suas próprias tarefas.
- Um utilizador comum apenas poderá alterar ou eliminar as suas próprias tarefas.
- Um administrador poderá consultar, alterar e eliminar todas as tarefas.
- Todos os endpoints de gestão de tarefas deverão exigir autenticação JWT.

Controlo de acessos

A API deverá distinguir corretamente os dois tipos de utilizadores.

Operação	Admin	Utilizador
Criar tarefa	Sim	Sim
Consultar todas as tarefas	Sim	Não
Consultar as próprias tarefas	Sim	Sim
Consultar tarefa de outro utilizador	Sim	Não
Alterar tarefa de outro utilizador	Sim	Não
Eliminar tarefa de outro utilizador	Sim	Não

Deverão ser utilizados mecanismos de proteção JWT e validação do tipo de utilizador.

Requisitos técnicos

O projeto deverá respeitar as seguintes condições:

1. Utilizar Flask para implementar a API.
2. Utilizar Flask-SQLAlchemy para gerir a base de dados.
3. Utilizar Flask-JWT-Extended para autenticação.
4. Utilizar SQLite como sistema de gestão de base de dados.
5. Utilizar SQLAlchemy Models para representar as tabelas.
6. Utilizar respostas HTTP adequadas, nomeadamente:
    - 200 – Pedido executado com sucesso.
    - 201 – Recurso criado.
    - 400 – Pedido inválido.

- 401 – Não autenticado.
- 403 – Sem permissões.
- 404 – Recurso não encontrado.

7. Organizar o código de forma estruturada e legível.

4. FORMATO E REGRAS DE ENTREGA

Formato:	**ZIP**	**Individual / Grupo:**	**Individual**
		Data de Entrega:	**13-09-2206**
Recursos / materiais necessários	**Conta no GitHub; Acesso ao CodeSpaces**		

5. CRITÉRIOS DE AVALIAÇÃO

Critério	Cotação
Cumprimento dos objetivos e das tarefas propostas	
Rigor técnico e correção dos conteúdos	
Organização, clareza e apresentação	
Criatividade, autonomia e cumprimento do prazo	

Observações do(a) professor(a): _____

Bom trabalho!